

Introduction to Image Processing Lab02 Report

EECS26 111060024 蔡孟伶

Project03_01

Implementation

In this project, the program inputs an image and outputs the log transformation and the power law transformation of the image according to the following transformation formula.

$$s = c \log(1 + r)$$

$$s = cr^\gamma$$

These operations transforms the original intensity of each pixel r to s by transformation T , i.e. $s = T(r)$. Since they addition, log function and scalar multiplication could be operated directly to the image element by element. I can implement the formula directly without complicated syntax, so it's straightforward.

Result Image

Following are the result image of the log transformation, original image, and $r = 0.8, 0.6, 0.4, 0.3, 0.2$ with $c = 1$ in all power transformation cases.



power-law transformation with $r = 0.60$



power-law transformation with $r = 0.40$



power-law transformation with $r = 0.30$



power-law transformation with $r = 0.20$





Discussion

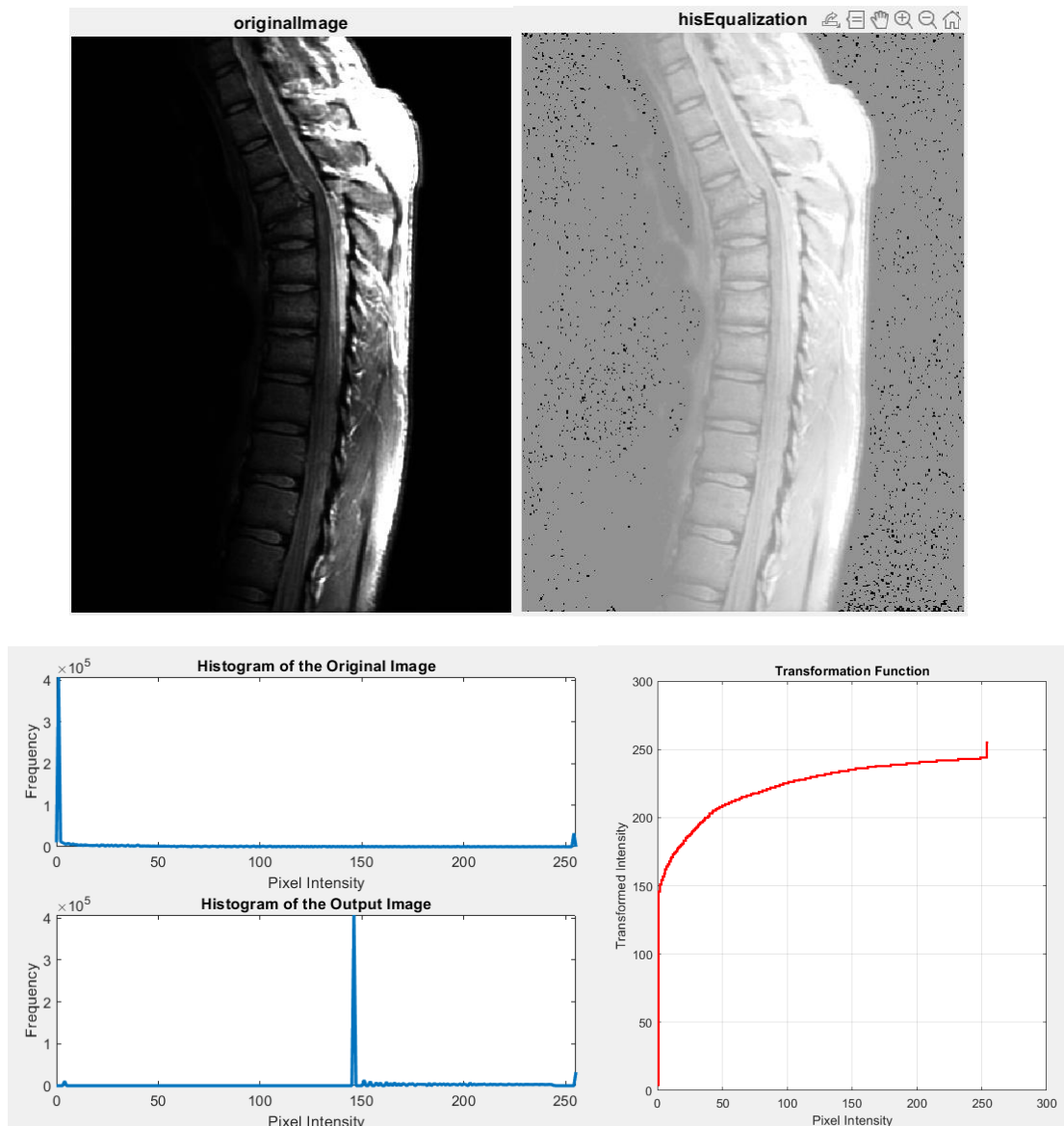
From the image above and according to the property of power function, we can see that when as gamma gets lower, the image gets brighter. On the other hand, if gamma is bigger than 1 and gets higher, it brightens the bright area and darkens the dark area. What log transformation do is to brighten the dark area and compresses the brightness of bright area, cause that is the property of log function. Both power-law function and log function could help to take a clearer look at the image's detail if most of it is dark.

Project03_02

Implementation

In this project, the function `imageHist` is supposed to return back the Histogram of the input image, after that function `histEqualization` is going to transform from the original histogram to a new, more balanced histogram, then mapped the intensity level back again. For every pixel intensity value, I counted the total number of it, and divided by the total number of pixels to get the pdf function. Then calculate the cdf based on the pdf and times it by $L - 1$, which is 255 in the grayscale picture. Finally, a mapping transformation is created and the output picture is done.

Result Image



Discussion

As figure 3.8 is an image whose intensity level is concentrated in dark region, we hope that this transformation outputs a image with appropriate brightness and balanced intensity.

After getting the output image from the Histogram equalization, we see that the output image is a little bit different from expected. This is actually because that the original image has most pixels in the low intensity region. It's too many such that when doing cumulative distribution function, it gets intense increase at some point easily. Therefore, the transformation function seems to do nothing at the beginning

but suddenly rise up. The overall intensity is concentrated in the region from gray to white instead of distributing perfectly from black to white.

However, this Histogram transformation still makes the detail of the image clearer than before processed. The whole image has somehow more concentrated intensity level in the middle too, according to the Histogram of the output image.

Project03_03, 04

Implementation

In this project, there are two functions that should be implemented. The first one is spatial filtering, which calculates the convolution of the mask and the image. The second one is the Laplacian filtering that calculates the second order difference by convolution with a special mask. Since the build-in function of convolution is forbidden, I make the convolution manually with double for loop and element-wise multiplication and summation. For the Laplacian filtering function, the formula follows by

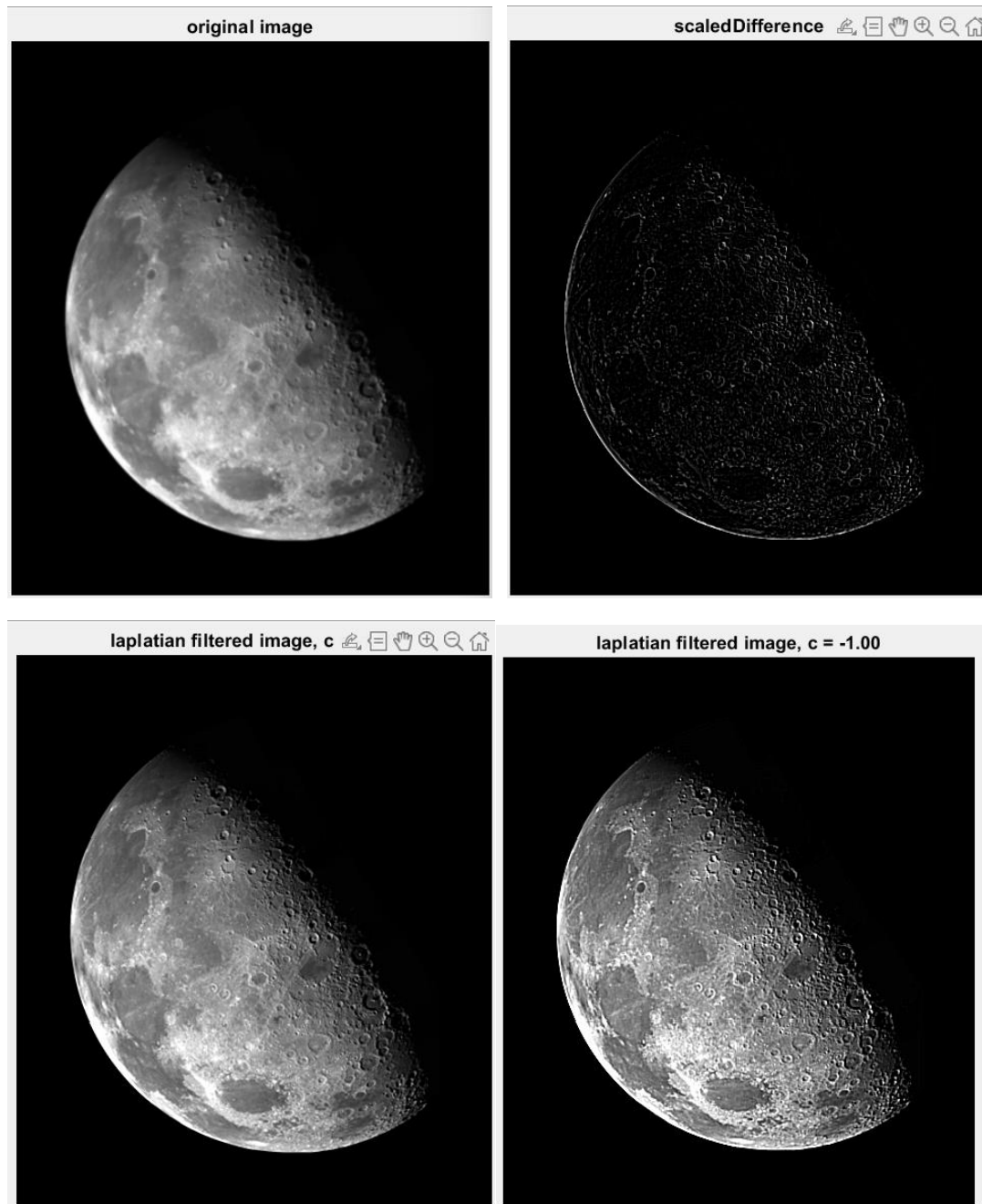
$$g(x, y) = f(x, y) + c[\nabla^2 f(x, y)]$$

To sharpen and enhance the edge of regions inside the image, c has to be a negative number. In this following result picture, different scales of c 's output picture would be shown. The kernel of Laplacian could be

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \text{ or } \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Result Image

Below are the reproduce of Fig 3.46(a, b, c, d) with a as the original picture, b being the Laplacian image, c being the sharpened image as $c = -1$ (using the kernel on the left), and d using the different Laplacian kernel(kernel on the right).

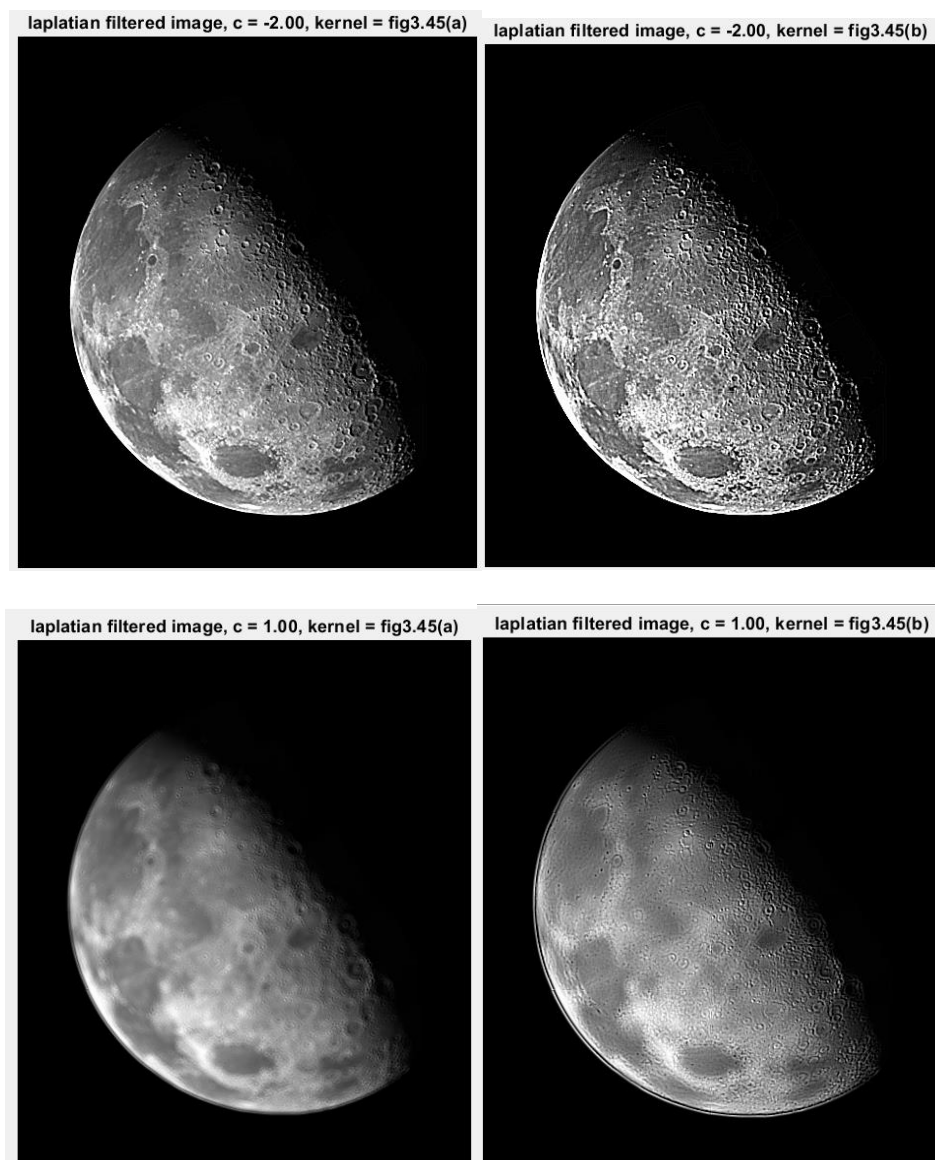


Discussion

In discussion we can discuss the output of different mask size and scale c , along with different strategies of the edge of the image during convolution. Before that, I want to introduce some different policies of padding of the image. In the program I implemented, I take zero-padding because it is easy and fast to implement. And the side effect of zero-padding is having a dark edge around the whole output image, since my image's background is already dark, it not easy to recognize. The other commonly used strategy used is the mirror padding. The padding pixels' value are then copied from the mirror of the boundaries, mirror paddings don't have problem

with dark edge, and it kind of smoothen the transition. Mirror padding deals better with convolution.

Now below are some output image with different scale and mask size. I won't repeat the result with $c = -1.00$, but in the two picture below, it is obvious that when $c = -2.00$, the contradistinction of the edge between bright and dark become bigger. When $c = 1.00$, you could see that the edge is detected though, but due to the wrong scale, it becomes somehow weird. At first glance, the image seemed a little blurred, but then you notice that it's only blurred at the edge.



Finally, we have the output from different mask size. The Laplacian kernel is showed below. For a larger Laplacian mask, it should be considering larger region on the original image and count for the edge. However, if the mask is too big, the output

ended up losing detail although the edge is already enhanced. For the following picture, I have $c = 1 / 16$ and $1 / 28$, it turned out that they are a little bit dark. The effect of sharpening is not as obvious as kernel that is $3 * 3$, however, if you take a closer look, you could see that the edge is enhanced, indeed. I think it's also important to find the correct scale for each mask size so that the sharpening could show well.

						0	0	0	-1	0	0	0
						0	0	-1	-2	-1	0	0
						0	-1	-2	-3	-2	-1	0
						-1	-2	-3	28	-3	-2	-1
						0	-1	-2	-3	-2	-1	0
						0	0	-1	-2	-1	0	0
						0	0	0	-1	0	0	0

and

0	0	-1	0	0
0	-1	-2	-1	0
-1	-2	16	-2	-1
0	-1	-2	-1	0
0	0	-1	0	0

